

Iliad D.2 — Vanilla Policy Gradient & Goal Misgeneralisation

Worked Solutions, Week 2 Day 2 (2026-06-16)

Beginner-friendly derivations with boxed key results and runnable code.
Grounded in the morning Vanilla Policy Gradient transcript + the goal-misgeneralisation literature.

What today asks you to do

Coding day, two halves (each: ~20-min talk + Colab exercises, solutions provided):

1. **Vanilla Policy Gradient:** derive the policy gradient; implement REINFORCE; add reward-to-go; add a baseline/advantage; train on CartPole.
2. **Goal Misgeneralisation:** build a proxy-vs-goal probe (CoinRun-style); show the agent pursues the proxy OOD; fix it with a diverse training distribution.

1 Vanilla Policy Gradient

1.1 Objective

MDP with policy $\pi_\theta(a \mid s)$, trajectory $\tau = (s_0, a_0, s_1, \dots)$, return $G(\tau) = \sum_t r(s_t, a_t)$. We maximise

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)], \quad P(\tau \mid \theta) = \rho_0(s_0) \prod_t \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

1.2 The policy gradient theorem (key derivation)

We cannot push ∇_θ through the expectation because θ parametrises the sampling distribution. Use the *log-derivative trick* $\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta$:

$$\nabla_\theta J = \nabla_\theta \int P(\tau \mid \theta) G(\tau) d\tau = \int P(\tau \mid \theta) \nabla_\theta \log P(\tau \mid \theta) G(\tau) d\tau = \mathbb{E}_\tau[\nabla_\theta \log P(\tau \mid \theta) G(\tau)].$$

Expanding $\log P$, the start distribution and the environment transitions carry *no* θ , so they vanish under ∇_θ :

$$\log P(\tau \mid \theta) = \log \rho_0(s_0) + \sum_t [\log \pi_\theta(a_t \mid s_t) + \log P(s_{t+1} \mid s_t, a_t)].$$

$$\nabla_\theta J = \mathbb{E}_\tau \left[\left(\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right) G(\tau) \right]$$

The magic: no dynamics model is needed — only $\nabla_\theta \log \pi_\theta$.

1.3 REINFORCE estimator and surrogate loss

Sample N trajectories and average. Equivalently minimise

$$L(\theta) = -\frac{1}{N} \sum_i \sum_t \log \pi_\theta(a_t^i | s_t^i) G(\tau^i).$$

The minus sign is the #1 bug: ascent on reward = descent on $-\log \pi \cdot G$.

1.4 Reward-to-go (variance reduction by causality)

An action at time t cannot affect earlier rewards, so replace $G(\tau)$ with $\hat{G}_t = \sum_{t' \geq t} r(s_{t'}, a_{t'})$:

$$\nabla_\theta J = \mathbb{E} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \hat{G}_t \right].$$

1.5 Baselines and the advantage

Any $b(s_t)$ independent of the action is unbiased, because $\mathbb{E}[\nabla_\theta \log \pi(a | s) b(s)] = b(s) \nabla_\theta \sum_a \pi(a | s) = b(s) \nabla_\theta 1 = 0$. Best simple choice: $b = V(s_t)$, giving the advantage $A(s_t, a_t) = \hat{G}_t - V(s_t)$:

$$\nabla_\theta J = \mathbb{E} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) A(s_t, a_t) \right].$$

VPG = this, with V regressed onto reward-to-go. PPO later clips this surrogate.

1.6 Implementation (CartPole)

```
import torch, torch.nn as nn, gymnasium as gym
from torch.distributions import Categorical

class Policy(nn.Module):
    def __init__(self, obs_dim, n_act, h=64):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(obs_dim, h), nn.Tanh(),
                                nn.Linear(h, h), nn.Tanh(), nn.Linear(h, n_act))
    def forward(self, x): return self.net(x) # logits

def reward_to_go(rews):
    rtg=[0.0]*len(rews); run=0.0
    for t in reversed(range(len(rews))):
        run=rews[t]+run; rtg[t]=run
    return rtg

def train(env_id="CartPole-v1", epochs=50, batch_steps=2000, lr=1e-2):
    env=gym.make(env_id)
    pi=Policy(env.observation_space.shape[0], env.action_space.n)
    opt=torch.optim.Adam(pi.parameters(), lr=lr)
    for ep in range(epochs):
        obs,act,w,rets=[],[],[],[]; steps=0
        while steps<batch_steps:
            o,_=env.reset(); er=[]; done=False
            while not done:
```

```

        a=Categorical(logits=pi(torch.as_tensor(o,dtype=torch.float32))).sample().item()
        obs.append(o); act.append(a)
        o,r,term,trunc,_=env.step(a); done=term or trunc
        er.append(r); steps+=1
        w+=reward_to_go(er); rets.append(sum(er))
    logp=Categorical(logits=pi(torch.as_tensor(obs,dtype=torch.float32)))
        ).log_prob(torch.as_tensor(act))
    wt=torch.as_tensor(w,dtype=torch.float32)
    loss=-(logp*wt).mean() # policy-gradient surrogate
    opt.zero_grad(); loss.backward(); opt.step()
    print(ep, sum(rets)/len(rets))

```

Gotchas tested by the Colab: weights w must be detached (only $\log \pi$ carries gradient); loss is negative; normalising w per batch is a cheap baseline; an entropy bonus $-\beta H[\pi]$ keeps exploration. Expected: CartPole return $\rightarrow 500$ within ~ 50 epochs.

2 Goal Misgeneralisation

2.1 Definition and the distinction that matters

Goal misgeneralisation (GMG): the agent stays *capable* out of distribution but optimises an *unintended goal* — with a *correct* reward on the training distribution. It is neither capability failure (it stays competent) nor reward misspecification (the reward was right).

2.2 Why

A *proxy* feature correlates with the true objective in training; SGD has no reason to prefer the true goal over the proxy. At deployment the proxy and the goal *decorrelate*, and a proxy-latching policy now confidently does the wrong thing. Causes: (i) training distribution not diverse enough; (ii) the proxy is simpler to represent than the true goal.

2.3 Canonical examples

- **CoinRun** (Langosco et al. 2021): coin always at the right end in training $\Rightarrow$ agent learns “go right”, not “get coin”. Move the coin $\Rightarrow$ it runs past it. Capable, wrong goal.
- **Keys/chests; follow-the-expert** gridworlds; **cultural-transmission** agents that keep imitating OOD.

2.4 Minimal probe (drop into the Colab gridworld)

```

# Train: coin ALWAYS rightmost column. Test: coin random column (proxy broken).
def evaluate_gmg(agent, test_envs):
    coin=proxy=0
    for env in test_envs:
        tr=rollout(agent, env)
        coin += tr.ended_on(env.coin_pos)
        proxy += tr.ended_on(env.rightmost_column)
    n=len(test_envs); return coin/n, proxy/n # GMG <=> proxy >> coin

```

Fix-it exercise: retrain with randomised coin position (diverse distribution). `reaches_coin` jumps, `reaches_proxy` drops — diversity that *breaks the proxy* is the first-line mitigation.

Takeaway. A correct reward is *not* sufficient for a correctly-generalising goal: the inductive bias of SGD plus a limited training distribution can install a competent agent with the wrong objective. This is a core reason “just specify the right reward” does not solve alignment.

Links to Iliad

VPG (§1) is the very optimiser that installs the misgeneralised goal (§2): same algorithm, capability vs alignment. GMG mirrors the Week-1 goal-directedness / coherence thread (Day-5 `representation_coherence_selection`).

References. Langosco et al. 2021 (arXiv:2105.14111); Shah et al. 2022 (arXiv:2210.01790); OpenAI Spinning Up “Vanilla Policy Gradient”; Sutton & Barto Ch.13; ARENA chapter2 (RL).